

① Compare Hash Table with separate chaining  
vs Binary Search Tree for a Dictionary

| Feature              | Hash Table - Separate chaining                                            | Binary search tree (BST)                                          |
|----------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------|
| Average search       | $O(1)$                                                                    | $O(\log n)$ if balanced                                           |
| Worst-case search    | $O(n)$                                                                    | $O(n)$ for unbalanced BST                                         |
| Locking granularity  | Can use per-bucket locks, allowing high concurrency                       | Can use node / subtree locks, but synchronization is more complex |
| Concurrent access    | Different keys mapped to different buckets can be accessed simultaneously | Operations may traverse common ancestors, causing more contention |
| Insertion / Deletion | Simple under bucket-level locking                                         | Requires careful synchronization, especially during rotation      |
| Resizing             | Expensive                                                                 | No global resizing required                                       |

Hash Tables with separate chaining generally support more scalable concurrent dictionary operations, especially when fine-grained per-bucket locking is used. They provide average  $O(1)$  lookup and allow independent buckets to be accessed concurrently. A BST is preferable when the dictionary requires ordered keys, range queries, or sorted traversal.



② Compare counting sort vs Radix sort vs Bucket sort for Uniform Floating-point numbers. Assume 100 million floating-point numbers uniformly distributed in  $[0.0, 1.0)$

| Feature                  | Counting Sort                      | Radix Sort                                               | Bucket Sort                            |
|--------------------------|------------------------------------|----------------------------------------------------------|----------------------------------------|
| Works directly on float  | No, requires mapping               | Not directly, float converted to sortable representation | Yes                                    |
| Distribution sensitivity | low after discretization           | Relatively low                                           | Very suitable for uniform distribution |
| Time complexity          | $O(n+k)$                           | $O(d(n+b))$                                              | $O(n)$                                 |
| Memory                   | $O(k)$                             | $O(n+k)$                                                 | $O(n + \text{no. of buckets})$         |
| Stability                | Stable if implemented properly     | Can be stable                                            | Stable depending on sorting            |
| Float handling           | Requires mapping float to integers | Requires bit-level transformation                        | Naturally divides $[0,1)$              |
| Main issue               | Huge range if precision is high    | More passes and complex float                            | Performance depends on distribution    |

Bucket sort is the most suitable choice for 100 million uniformly distributed floating-point numbers in  $[0.0, 1.0)$  as :-

- \* It handles floating-point values naturally
- \* It avoids the huge counting array required by counting sort
- \* It requires fewer processing passes than radix sort
- \* The algorithm can be parallelized by assigning different buckets to different threads